TP 3: Information Retrieval Models
(Boolean Retrieval and the Inverted Index)

Problem Set:

Learn how to represent a set of documents with a term-document incidence matrix and an inverted index, and apply the Boolean retrieval model to answer AND, OR, NOT and phrase queries by merging postings lists.

Problem 1: Create a Term-Document Incidence Matrix

Task: Create a binary term-document incidence matrix for a small set of documents.

Instructions:

1. Choose the following six sample documents (docIDs 1 to 6):
    - Document 1: "Data science combines statistics, computer science, and domain knowledge."
    - Document 2: "Machine learning algorithms can analyze large datasets and make predictions."
    - Document 3: "Data visualization helps in interpreting complex data and communicating insights."
    - Document 4: "Computer science students learn machine learning and data mining."
    - Document 5: "Search engines use an inverted index to retrieve documents for a query."
    - Document 6: "Statistics and machine learning help data scientists build predictive models."
2. Write a function to tokenize each document (convert to lowercase, split into words, remove punctuation) and build the sorted vocabulary.
3. Construct the incidence matrix: rows represent terms, columns represent documents, and an entry is 1 if the term occurs in the document, 0 otherwise. Print it with labels for terms and documents.
4. Answer the query **data AND science AND NOT machine** with bitwise operations on the rows of the matrix: take the complement of the row of *machine*, then AND the three vectors (Lecture 3: *Brutus AND Caesar AND NOT Calpurnia*). Print the vectors and the matching documents.

Problem 2: Build an Inverted Index

Task: Build the inverted index of the collection by sorting (term, docID) pairs.

Instructions:

1. Using the tokenizer from Problem 1, generate the sequence of (term, docID) pairs for all documents, in document order.
2. Sort the pairs by term, then by docID.

3. Merge multiple entries of the same term in the same document, then group the pairs into a dictionary: term → (document frequency, postings list). Each postings list must be sorted by docID.
4. Print the document frequency and the postings list of: *data*, *science*, *machine*, *learning*, *statistics*, *computer*.
5. Compare the number of cells of the incidence matrix with the number of postings in the index. Which representation would you use for a collection of 1 million documents, and why?

Problem 3: Implement the Merge Algorithm

Task: Process Boolean queries by merging the postings lists of the inverted index.

Instructions:

1. Using the index from Problem 2, implement the INTERSECT algorithm of Lecture 3, which walks through the two sorted postings lists simultaneously in O(x + y) time:

```
INTERSECT(p1, p2)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then ADD(answer, docID(p1))
 5             p1 ← next(p1)
 6             p2 ← next(p2)
 7        else if docID(p1) < docID(p2)
 8                then p1 ← next(p1)
 9                else p2 ← next(p2)
10  return answer
```

2. Write two more functions for **x OR y** and **x AND NOT y**, also as linear merges of the two sorted lists (do not use Python sets).
3. Process the following queries and print the results:
    - Query 1: data AND machine
    - Query 2: statistics OR learning
    - Query 3: data AND NOT science

Problem 4: Query Optimization

Task: Choose the order in which the postings lists of a query are merged.

Instructions:

1. Write a function that processes a conjunctive query t_1 AND t_2 AND … AND t_n by intersecting the postings lists in **increasing order of document frequency** (start with the shortest list), and counts the number of docID comparisons.
2. Process the query **data AND machine AND science** twice: once in the order given and once in the optimized order. Print the result and the number of comparisons for each order.

3. For the query **(statistics OR learning) AND (data OR computer) AND (science OR models)**, estimate the size of each OR by the sum of the document frequencies of its terms, then process the three groups in increasing order of their estimates. Print the estimates, the processing order and the result.

Problem 5: Phrase Queries with a Positional Index

Task: Extend the inverted index with term positions to answer phrase queries.

Instructions:

1. Build a positional index: for each term, store each document that contains it together with the list of positions of the term in that document (term → {docID: [positions]}).
2. Write a function to answer a phrase query: a document matches if the terms of the phrase occur at consecutive positions (position p, $p + 1$, …).
3. Process the following phrase queries and print the results:
    - Query 1: "machine learning"
    - Query 2: "computer science"
    - Query 3: "data science"
4. Compare the result of "data science" with the Boolean query **data AND science** from Problem 3. Which document is returned by one query but not the other, and why?